

De acuerdo. Para el **Reporte 1 — EDA / Data Readiness** vamos a trabajar la población analítica como **un único dataset**, sin mencionar todavía ningún esquema de partición temporal para modelado. Tampoco usaremos “victim/non-victim”; hablaremos de **positive/event** y **negative/non-event**, porque el outcome es la primera alerta calificante observada.

La fuente principal para el EDA puede ser:

```
bu_frd.model_features_ab_final_2022_2025
```

porque ya representa la población final a nivel **party_id**, y para controles de integridad del outcome utilizaremos también:

```
bu_frd.model_cohort_behavioral_final_3m_2022_2025  
bu_frd.party_first_valid_alert_all
```

Empecemos por EDA 1 — Dataset Overview & Class Balance

Primero necesitamos congelar las métricas básicas del universo completo. Ejecuta esto en R/Spark:

```
eda_class_balance <- sdf_sql(  
  sc,  
  "  
  SELECT  
    COUNT(*) AS total_records,  
    COUNT(DISTINCT party_id) AS unique_parties,  
  
    SUM(CASE WHEN label_3m = 1 THEN 1 ELSE 0 END) AS positive_cases,  
    SUM(CASE WHEN label_3m = 0 THEN 1 ELSE 0 END) AS negative_cases,  
  
    AVG(CASE WHEN label_3m = 1 THEN 1.0 ELSE 0.0 END) AS positive_rate,  
    AVG(CASE WHEN label_3m = 0 THEN 1.0 ELSE 0.0 END) AS negative_rate,  
  
    SUM(CASE WHEN label_3m = 0 THEN 1 ELSE 0 END) * 1.0 /  
    SUM(CASE WHEN label_3m = 1 THEN 1 ELSE 0 END) AS negative_to_positive_ratio  
  
  FROM bu_frd.model_features_ab_final_2022_2025  
  "  
) %>%  
  collect()  
  
eda_class_balance
```

De acuerdo con los controles que ya tenemos, deberíamos quedar alrededor de:

Metric	Expectad
Total records	688,589
Unique parties	688,589
Positive cases	62,599
Negative cases	625,990
Positive rate	9.09%
Negative rate	90.91%
Negative : Positive	10 : 1

Pero para el reporte final utilizaremos **los resultados que arroje el query**, no números escritos manualmente.

Hay una consideración importante para la narrativa del EDA: ese **9.09%** es la composición de nuestra **muestra analítica**. No debemos presentarlo como prevalencia poblacional de fraud alerts.

EDA 2 — Basic Dataset Integrity

Antes de cualquier gráfico de comportamiento, confirmemos que la unidad analítica está limpia:

```
eda_basic_integrity <- sdf_sql(
  sc,
  "
  SELECT

    COUNT(*) AS total_rows,

    COUNT(DISTINCT party_id) AS distinct_parties,

    COUNT(*) - COUNT(DISTINCT party_id) AS duplicate_party_rows,

    SUM(
      CASE
        WHEN party_id IS NULL THEN 1
        ELSE 0
      END
    ) AS null_party_id,
```

```

SUM(
  CASE
    WHEN observation_date IS NULL THEN 1
    ELSE 0
  END
) AS null_observation_date,

SUM(
  CASE
    WHEN label_3m IS NULL THEN 1
    ELSE 0
  END
) AS null_label,

SUM(
  CASE
    WHEN label_3m NOT IN (0,1) THEN 1
    ELSE 0
  END
) AS invalid_label

FROM bu_frd.model_features_ab_final_2022_2025
"
) %>%
collect()

```

eda_basic_integrity

Idealmente queremos:

```

duplicate_party_rows = 0
null_party_id        = 0
null_observation_date = 0
null_label           = 0
invalid_label        = 0

```

Esto se convertirá posteriormente en una pequeña sección de **Data Integrity KPI cards**, no necesariamente en una gráfica.

EDA 3 — Outcome Integrity

Este QA es más importante que el simple null check. Queremos demostrar que el label realmente respeta la definición temporal:

```

\[ label=1 \iff observation\_date < first\_alert\_date \leq
LAST\_DAY(ADD\_MONTHS(observation\_date,3)) \]

```

Ejecuta:

```
eda_label_integrity <- sdf_sql(  
  sc,  
  "  
  WITH base AS (  
  
    SELECT  
      party_id,  
      observation_date,  
      label_3m  
  
    FROM bu_frd.model_features_ab_final_2022_2025  
  
  ),  
  
  alerts AS (  
  
    SELECT  
      party_id,  
      first_alert_date  
  
    FROM bu_frd.party_first_valid_alert_all  
  
  )  
  
  SELECT  
  
    COUNT(*) AS total_rows,  
  
    SUM(  
      CASE  
        WHEN b.label_3m = 1  
          AND a.first_alert_date IS NULL  
        THEN 1 ELSE 0  
      END  
    ) AS positive_without_alert,  
  
    SUM(  
      CASE  
        WHEN b.label_3m = 1  
          AND a.first_alert_date <= b.observation_date  
        THEN 1 ELSE 0  
      END  
    ) AS positive_alert_not_future,  
  
    SUM(  
      CASE
```

```

        WHEN b.label_3m = 1
        AND a.first_alert_date >
            LAST_DAY(ADD_MONTHS(b.observation_date,3))
        THEN 1 ELSE 0
    END
) AS positive_alert_outside_horizon,

SUM(
    CASE
        WHEN b.label_3m = 0
        AND a.first_alert_date > b.observation_date
        AND a.first_alert_date <=
            LAST_DAY(ADD_MONTHS(b.observation_date,3))
        THEN 1 ELSE 0
    END
) AS negative_with_alert_in_horizon

FROM base b

LEFT JOIN alerts a
    ON b.party_id = a.party_id
"
) %>%
collect()

eda_label_integrity

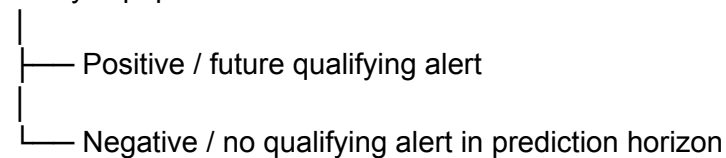
```

Aquí idealmente todos estos errores deben ser 0.

Primera gráfica del EDA

Con esta primera sección podemos construir después una gráfica muy sencilla:

Analytic population



Para la presentación probablemente usaría **dos KPI cards + una barra de composición**, no un pie chart, porque la diferencia 10:1 se compara mejor visualmente con barras.

También podemos incorporar un KPI:

Unique party coverage = 100%

si confirmamos:

$$TotalRecords = UniqueParties$$

2-Siguiente sección: Temporal Cohort Profile

Sin hablar todavía de particiones de modelado, debemos revisar cómo está distribuida la población por `observation_date`. Esto nos permite detectar concentraciones, huecos o cambios fuertes a través del tiempo.

Primero, resumen anual:

```
eda_cohort_year <- sdf_sql(
  sc,
  "
  SELECT
    YEAR(observation_date) AS observation_year,

    COUNT(*) AS total_parties,

    SUM(CASE WHEN label_3m = 1 THEN 1 ELSE 0 END) AS positive_cases,

    SUM(CASE WHEN label_3m = 0 THEN 1 ELSE 0 END) AS negative_cases,

    AVG(CASE WHEN label_3m = 1 THEN 1.0 ELSE 0.0 END) AS positive_rate

  FROM bu_frd.model_features_ab_final_2022_2025

  GROUP BY YEAR(observation_date)

  ORDER BY observation_year
  "
) %>%
  collect()

eda_cohort_year
```

Y después a nivel mensual:

```
eda_cohort_month <- sdf_sql(
  sc,
  "
  SELECT
    YEAR(observation_date) AS observation_year,
    MONTH(observation_date) AS observation_month,

    COUNT(*) AS total_parties,

    SUM(CASE WHEN label_3m = 1 THEN 1 ELSE 0 END) AS positive_cases,
```

```

SUM(CASE WHEN label_3m = 0 THEN 1 ELSE 0 END) AS negative_cases,

AVG(CASE WHEN label_3m = 1 THEN 1.0 ELSE 0.0 END) AS positive_rate

FROM bu_frd.model_features_ab_final_2022_2025

GROUP BY
  YEAR(observation_date),
  MONTH(observation_date)

ORDER BY
  observation_year,
  observation_month
"
) %>%
collect()

```

eda_cohort_month

Y un pequeño control del rango:

```

eda_date_range <- sdf_sql(
  sc,
  "
SELECT
  MIN(observation_date) AS min_observation_date,
  MAX(observation_date) AS max_observation_date,
  COUNT(DISTINCT observation_date) AS distinct_observation_dates

FROM bu_frd.model_features_ab_final_2022_2025
"
) %>%
collect()

```

eda_date_range

Con estos resultados podremos construir dos visuales posteriores:

Population volume over calendar time

Positive rate over calendar time

La segunda será particularmente importante: no queremos asumir que la tasa analítica es constante a través del tiempo.

Después de esta sección avanzaría a **Tenure / observation-history confounding**, pero primero quiero ver estos resultados temporales para no mezclar posibles efectos de calendario con tenure.

3-Siguiente sección: Tenure / Observation-History Confounding

Aquí sí tenemos que ser cuidadosos porque previamente identificamos que la fuente utilizada para tenure está expuesta a **left censoring**. Por eso no quiero construir todavía buckets arbitrarios sin primero conocer la distribución de la variable disponible.

El siguiente paso debería ser primero identificar qué información de tenure quedó disponible en nuestra cohorte final. Ejecutemos:

```
eda_tenure_profile <- sdf_sql(  
  sc,  
  "  
  SELECT  
    COUNT(*) AS total_parties,  
  
    SUM(CASE WHEN tenure_months IS NULL THEN 1 ELSE 0 END)  
    AS tenure_null_count,  
  
    AVG(tenure_months) AS avg_tenure_months,  
  
    MIN(tenure_months) AS min_tenure_months,  
  
    percentile_approx(tenure_months, 0.01) AS p01,  
    percentile_approx(tenure_months, 0.05) AS p05,  
    percentile_approx(tenure_months, 0.25) AS p25,  
    percentile_approx(tenure_months, 0.50) AS p50,  
    percentile_approx(tenure_months, 0.75) AS p75,  
    percentile_approx(tenure_months, 0.95) AS p95,  
    percentile_approx(tenure_months, 0.99) AS p99,  
  
    MAX(tenure_months) AS max_tenure_months  
  
  FROM bu_frd.model_features_ab_final_2022_2025  
  "  
) %>%  
  collect()
```

eda_tenure_profile

Pero antes de ejecutarlo, verifica:

```
colnames(  
  tbl(sc, "bu_frd.model_features_ab_final_2022_2025")  
)
```

porque no quiero asumir que `tenure_months` terminó físicamente dentro de la tabla final. Si no aparece, **no ejecutes el primer query**. Pásame los nombres relacionados con `tenure/person history` que aparezcan, o simplemente dime que no existe.

Esto es intencional: para el EDA prefiero volver a la fuente correcta de `tenure` si es necesario, en lugar de reconstruir una métrica con una columna equivocada.

Estado del EDA hasta ahora

Tenemos ya dos bloques que pueden considerarse cerrados:

Dataset Structure & Outcome Integrity — PASS

- 688,589 observaciones.
- 688,589 parties únicos.
- 0 duplicados.
- 0 labels inválidos.
- 0 violaciones del horizonte temporal del outcome.

Temporal Coverage — PASS, with cohort-volume concentration noted

- 48 meses consecutivos.
- Enero 2022 → diciembre 2025.
- Sin gaps mensuales.
- Variación importante del volumen analítico.
- Concentración particularmente alta en mayo–junio 2024.
- Positive rate constante por diseño de sampling, por lo que **no se utilizará como evidencia de estabilidad temporal**.

Este último punto es una mejora importante respecto al EDA framework original.

3.1.

Perfecto. Eso confirma que `tenure_months` **no forma parte del dataset final de features**, y está bien: para este EDA podemos tratar `tenure` como una **variable auxiliar de diagnóstico**, no como predictor del modelo.

Además, esto evita modificar `bu_frd.model_features_ab_final_2022_2025` solo para hacer el reporte.

El siguiente paso es volver a la fuente donde evaluamos `tenure`, `bc3.person_restricted`, pero antes necesito que confirmemos exactamente qué columnas de fechas están disponibles actualmente. Ejecuta únicamente este diagnóstico:

```
person_restricted_cols <- colnames(  
  
tbl(sc, "bc3.person_restricted")
```

)

person_restricted_cols

Si la lista es muy larga, puedes usar:

person_restricted_cols[

grepl(

"date|dte|create|start|open|first|party",

person_restricted_cols,

ignore.case = TRUE

)

]

También puedes obtener schema y tipos:

person_restricted_schema <- sdf_sql(

sc,

"

DESCRIBE bc3.person_restricted

"

) %>%

collect()

person_restricted_schema

No construiría todavía **tenure_months** hasta ver cuál es exactamente la fecha disponible.

Por qué lo hacemos separado

Nuestro EDA quedaría conceptualmente:

FINAL ANALYTIC COHORT

bu_frd.model_features_ab_final_2022_2025

|
| party_id



TENURE AUXILIARY SOURCE

bc3.person_restricted

Luego calcularemos algo conceptualmente parecido a:

$$TenureMonths = months_between(observation_date, first_observed_customer_date)$$

pero **solo después de verificar qué representa realmente esa fecha.**

Esto es importante porque anteriormente encontramos que esta fuente tiene un problema potencial de **left censoring**. Es decir, para algunos clientes la primera fecha observada puede representar simplemente el inicio de nuestra disponibilidad histórica y no necesariamente el verdadero inicio de su relación.

Por eso en el reporte no deberíamos decir:

“Customer tenure is X months”

sin más.

Sería más correcto llamarlo algo como:

Observed tenure / available customer history

y separar, si es posible:

reliably observed tenure

vs

left-censored tenure

No necesitamos alterar el resto del EDA

Hasta ahora seguimos teniendo cerrados:

- | | |
|-------------------------------------|------|
| 1. Dataset Overview & Class Balance | PASS |
| 2. Dataset Integrity | PASS |
| 3. Outcome Integrity | PASS |

4. Temporal Coverage PASS

Estamos únicamente resolviendo:

5. Tenure / Observed-History Confounding

Una vez me pases las columnas relacionadas con fechas de `bc3.person_restricted`, te doy el query exacto para construir los buckets y comparar `count`, `positive count`, `positive rate`, `censoring` y `max spread`, sin incorporar tenure como feature del modelo.

3.2

Por lo tanto, para el EDA podemos analizar **observed customer history**, pero no llamarlo “true tenure”.

La definición correcta sería:

$$\text{ObservedTenureMonths} = \text{MonthsBetween}(\text{ObservationDate}, \text{FirstObservedDataDate})$$

donde:

$$\text{FirstObservedDataDate} = \min(\text{data_date})$$

por `party_id`.

El riesgo es **left censoring**: si muchos clientes aparecen por primera vez cerca del inicio histórico de `person_restricted`, ese `MIN(data_date)` no representa realmente cuándo comenzó la relación con el cliente.

Primero perfilaremos la fuente

Ejecuta:

```
eda_person_restricted_profile <- sdf_sql(  
  sc,  
  "  
  
  SELECT  
  
    COUNT(*) AS total_rows,  
  
    COUNT(DISTINCT party_id) AS distinct_parties,
```

```
MIN(data_date) AS min_data_date,  
MAX(data_date) AS max_data_date,  
SUM(CASE WHEN party_id IS NULL THEN 1 ELSE 0 END) AS null_party_id,  
SUM(CASE WHEN data_date IS NULL THEN 1 ELSE 0 END) AS null_data_date
```

```
FROM bc3.person_restricted
```

```
"
```

```
) %>%
```

```
collect()
```

```
eda_person_restricted_profile
```

Después construimos el primer registro observado por party y lo cruzamos contra nuestra población analítica:

```
eda_observed_tenure_profile <- sdf_sql(
```

```
sc,
```

```
"
```

```
WITH first_seen AS (
```

```
SELECT
```

```
party_id,
```

```
MIN(data_date) AS first_observed_date
```

```
FROM bc3.person_restricted
```

```
WHERE party_id IS NOT NULL
```

```
AND data_date IS NOT NULL
```

```
GROUP BY party_id  
)
```

```
cohort AS (
```

```
SELECT  
    party_id,  
    observation_date,  
    label_3m
```

```
FROM bu_frd.model_features_ab_final_2022_2025  
)
```

```
joined AS (
```

```
SELECT  
    c.party_id,  
    c.observation_date,  
    c.label_3m,  
    f.first_observed_date,
```

```
CASE  
    WHEN f.first_observed_date IS NOT NULL  
    THEN FLOOR(  
        MONTHS_BETWEEN(  
            c.observation_date,
```

```

        f.first_observed_date
    )
)
ELSE NULL
END AS observed_tenure_months

FROM cohort c

LEFT JOIN first_seen f
    ON c.party_id = f.party_id
)

SELECT
    COUNT(*) AS total_parties,

    SUM(
        CASE
            WHEN first_observed_date IS NULL THEN 1
            ELSE 0
        END
    ) AS missing_first_observed_date,

    MIN(first_observed_date) AS min_first_observed_date,

    MAX(first_observed_date) AS max_first_observed_date,

```

AVG(observed_tenure_months) AS avg_observed_tenure_months,

MIN(observed_tenure_months) AS min_observed_tenure_months,

percentile_approx(observed_tenure_months, 0.01) AS p01,

percentile_approx(observed_tenure_months, 0.05) AS p05,

percentile_approx(observed_tenure_months, 0.25) AS p25,

percentile_approx(observed_tenure_months, 0.50) AS p50,

percentile_approx(observed_tenure_months, 0.75) AS p75,

percentile_approx(observed_tenure_months, 0.95) AS p95,

percentile_approx(observed_tenure_months, 0.99) AS p99,

MAX(observed_tenure_months) AS max_observed_tenure_months

FROM joined

"

) %>%

collect()

eda_observed_tenure_profile

También necesitamos saber si existe acumulación artificial en la primera fecha disponible de la fuente. Este query es especialmente importante:

eda_first_seen_distribution <- sdf_sql(

sc,

"

WITH first_seen AS (

```

SELECT
    party_id,
    MIN(data_date) AS first_observed_date

FROM bc3.person_restricted

WHERE party_id IS NOT NULL
    AND data_date IS NOT NULL

GROUP BY party_id
),

cohort AS (

    SELECT DISTINCT party_id
    FROM bu_frd.model_features_ab_final_2022_2025
)

SELECT
    YEAR(f.first_observed_date) AS first_seen_year,
    COUNT(*) AS parties

FROM cohort c

INNER JOIN first_seen f
    ON c.party_id = f.party_id

```

```
GROUP BY YEAR(f.first_observed_date)
```

```
ORDER BY first_seen_year
```

```
"
```

```
) %>%
```

```
collect()
```

eda_first_seen_distribution

Con esos tres resultados decidiremos cómo hacer los buckets.

La decisión metodológica aquí será importante: si observamos una gran concentración en el primer año disponible, **no eliminaremos esta sección del EDA**, pero la presentaremos como:

Observed-history confound analysis

en vez de:

Customer tenure analysis.

Y probablemente separaremos:

Potentially left-censored

vs

more reliably observed

antes de calcular positive rates por duración observada.

Pásame los resultados de esos tres queries y definimos los buckets sin asumir arbitrariamente dónde empieza el censoring.

3.3

Ejecuta:

```
eda_source_start_censoring <- sdf_sql(
```

```
  sc,
```

```
  "
```

```
  WITH first_seen AS (
```

```
    SELECT
```

```
      pr.party_id,
```

```
      MIN(pr.data_date) AS first_observed_date
```

```
    FROM bc3.person_restricted pr
```

```
    INNER JOIN (
```

```
      SELECT DISTINCT party_id
```

```
      FROM bu_frd.model_features_ab_final_2022_2025
```

```
    ) c
```

```
    ON pr.party_id = c.party_id
```

```
    GROUP BY pr.party_id
```

```
  )
```

```
  SELECT
```

```
    COUNT(*) AS total_parties,
```

```

SUM(
  CASE
    WHEN first_observed_date = DATE '2000-09-30'
    THEN 1 ELSE 0
  END
) AS source_start_censored_parties,

```

```

AVG(
  CASE
    WHEN first_observed_date = DATE '2000-09-30'
    THEN 1.0 ELSE 0.0
  END
) AS source_start_censored_pct,

```

```

MIN(first_observed_date) AS min_first_seen,
MAX(first_observed_date) AS max_first_seen

```

```

FROM first_seen

```

```

"

```

```

) %>%

```

```

collect()

```

```

eda_source_start_censoring

```

Después sí construiría los buckets de **observable history**, no tenure:

```

eda_observed_history_buckets <- sdf_sql(

```

```

sc,

```

"

WITH first_seen AS (

SELECT

pr.party_id,

MIN(pr.data_date) AS first_observed_date

FROM bc3.person_restricted pr

INNER JOIN (

SELECT DISTINCT party_id

FROM bu_frd.model_features_ab_final_2022_2025

) c

ON pr.party_id = c.party_id

GROUP BY pr.party_id

),

base AS (

SELECT

c.party_id,

c.label_3m,

c.observation_date,

f.first_observed_date,

```

FLOOR(
    MONTHS_BETWEEN(
        c.observation_date,
        f.first_observed_date
    )
) AS observed_history_months,

CASE
    WHEN f.first_observed_date = DATE '2000-09-30'
    THEN 1
    ELSE 0
END AS source_start_censored_flag

FROM bu_frd.model_features_ab_final_2022_2025 c

LEFT JOIN first_seen f
    ON c.party_id = f.party_id
),

bucketed AS (

SELECT
    *,

CASE
    WHEN source_start_censored_flag = 1

```

```

        THEN 'SOURCE_START_CENSORED'
    WHEN observed_history_months < 24
        THEN '00-23 months'
    WHEN observed_history_months < 60
        THEN '24-59 months'
    WHEN observed_history_months < 120
        THEN '60-119 months'
    WHEN observed_history_months < 180
        THEN '120-179 months'
    WHEN observed_history_months < 240
        THEN '180-239 months'
    ELSE '240+ months'
END AS observed_history_bucket

FROM base
)

SELECT
    observed_history_bucket,

    COUNT(*) AS parties,

    SUM(CASE WHEN label_3m = 1 THEN 1 ELSE 0 END) AS positive_cases,

    AVG(CASE WHEN label_3m = 1 THEN 1.0 ELSE 0.0 END)
        AS positive_rate,

```

```

    AVG(observed_history_months)
      AS avg_observed_history_months

FROM bucketed

GROUP BY observed_history_bucket

ORDER BY

CASE observed_history_bucket
  WHEN '00-23 months' THEN 1
  WHEN '24-59 months' THEN 2
  WHEN '60-119 months' THEN 3
  WHEN '120-179 months' THEN 4
  WHEN '180-239 months' THEN 5
  WHEN '240+ months' THEN 6
  WHEN 'SOURCE_START_CENSORED' THEN 7
END
"
) %>%
collect()

```

eda_observed_history_buckets

Con eso podremos calcular el spread entre buckets **no censurados** y verificar si la longitud de historia observable está relacionada con el label.

Para el reporte final, esta sección se llamaría algo como:

Source History & Left-Censoring Review

